

Challenge: Build an Unstructured Data Ingestion Pipeline

Sep 25, 2026 · @Oviya

Overview

AI applications live or die by their data, and most useful data is locked inside messy formats: PDFs full of tables and charts, and web pages full of images and layout noise. This challenge asks you to build a working prototype that pulls content out of those sources, converts it into clean, standardized Markdown, stores it in an organized way in the cloud, and exposes the whole thing through an API and a simple web app.

Along the way you'll compare open-source tools against an enterprise document-processing service, so you finish with both a working pipeline and an informed opinion about which tools to use when.

The scenario: You've joined an early-stage startup building an AI product that depends on large amounts of data from PDFs and web pages. Before the team commits to a stack, they need you to test the options and build a prototype that proves the approach works.

Who this is for: Developers, data engineers, and ML practitioners comfortable with Python and basic cloud concepts. You can do it solo or as a team.

Suggested time: 1 week.

What you'll build

An end-to-end pipeline with four layers:

1. **Extraction:** pull text, tables, and images from PDFs and web pages, using both open-source libraries and an enterprise service.
2. **Standardization:** convert everything into consistent Markdown using Docling and MarkItDown.
3. **Storage:** organize outputs and image assets in an Amazon S3 bucket with a clear, searchable structure.
4. **Access:** a FastAPI backend and a hosted Streamlit front end so anyone can upload a PDF or paste a URL and get Markdown back.

Tool map

Source	Open-source extraction	Enterprise service	Markdown conversion
PDF	PyPDF2 / pypdf, pdfplumber	Azure AI Document Intelligence (or Adobe PDF Services, AWS Textract, etc.)	Docling, Markdown
Web page	requests + BeautifulSoup, Scrapy	Azure AI Document Intelligence, or a service of your choice	Docling, Markdown

You're free to swap in comparable tools. Just explain why in your write-up.

The tasks

Pick your test set before you start: at least 2–3 PDFs and 2–3 web pages. Pick hard, challenging PDFs from arXiv (arxiv.org), each at least 5 pages long. Easy, text-only papers won't tell you much about the tools.

Your PDFs must be truly multimodal. Each one should contain most or all of the following, so the tools are properly stress-tested:

- Headings and multi-column or mixed-layout text
- Tables (ideally one that spans columns or pages)
- Mathematical formulas or equations
- Key-value pairs (e.g. metadata blocks, parameter or hyperparameter lists, labeled results)
- Images, charts, and diagrams with captions

Where to look on arXiv: papers in math, physics, quantitative finance, or machine learning usually mix dense equations, multi-page tables, figures, and charts. Web pages should likewise include tables and images.

Task 1: Extract content from PDFs

- ☐ Write two Python functions that parse your PDFs using open-source tools (e.g. PyPDF2/pypdf for text, pdfplumber for tables and layout).
- ☐ Extract text, tables, and embedded images; save images as separate files.

- ☐ Process the same PDFs with an enterprise service (e.g. Azure AI Document Intelligence) and convert the result to Markdown.

Done when: each PDF produces open-source output and enterprise output you can compare side by side.

Task 2: Extract content from web pages

- ☐ Write two Python functions that scrape your chosen pages using requests + BeautifulSoup (or Scrapy).
- ☐ Capture text, tables, and images (download images or record their URLs).
- ☐ Process the same pages with an enterprise service and convert the result to Markdown.

Done when: each page produces comparable open-source and enterprise output. Respect each site's terms of use and robots.txt.

Task 3: Compare all three approaches

Write one comparison of all three approaches, run on the same test set: (1) open-source Python libraries (PyPDF2/pdfplumber, BeautifulSoup), (2) Docling and MarkItDown, and (3) the enterprise cloud service. You'll finish this after Task 4, once you have Docling and MarkItDown output. Cover:

- **Accuracy:** how well text, tables, and images survive extraction
- **Performance:** processing time per document
- **Ease of use:** setup effort, docs quality, amount of code needed
- **Cost:** free vs. per-page pricing at small and large volumes
- **Scalability and integration:** how each fits into a production pipeline

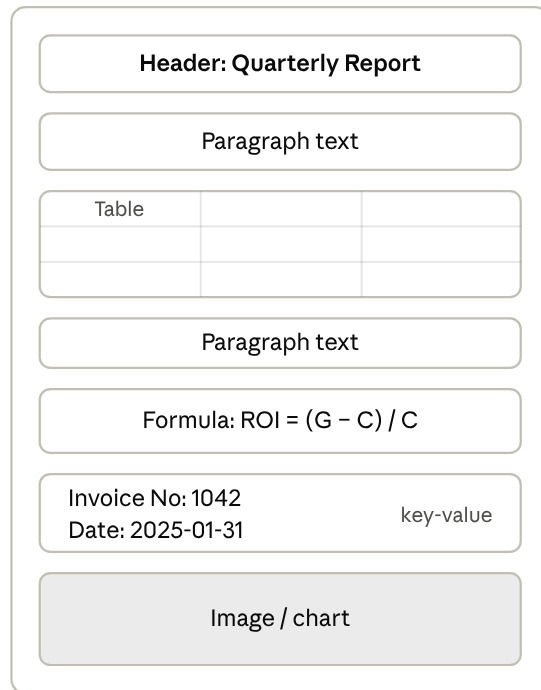
Done when: you can say which approach works best for which kind of content, and make a clear overall recommendation backed by examples from your test set.

Task 4: Reassemble and standardize with Docling and MarkItDown

Extraction gives you pieces: a block of text here, a table there, an image file, a formula. The goal of this task is to **put those pieces back together in the right place** and render one clean Markdown file that reads like the original document.

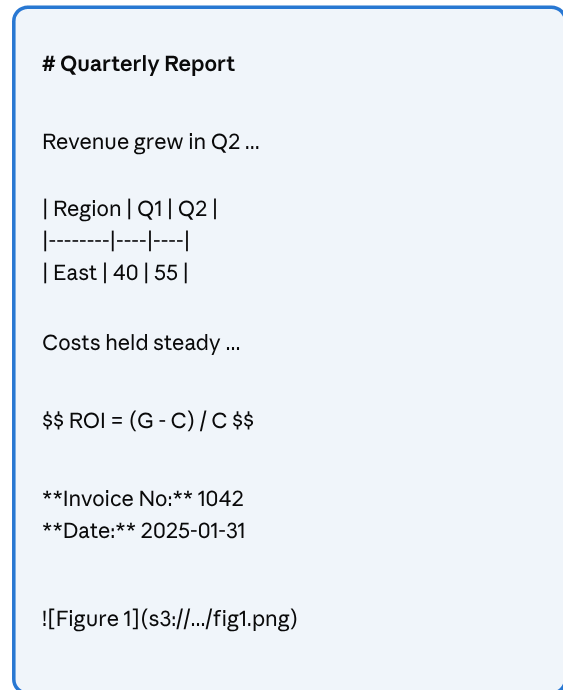
From PDF layout to reading-order Markdown

Source PDF (one page)



reassemble
in order

Final Markdown



one PDF page · six element types · reassembled into Markdown in reading order

Each element in the PDF maps to its Markdown equivalent, in the same reading order: headers become `#` headings, tables become Markdown tables, formulas become LaTeX (`$$ ... $$`), key-value pairs become bold labels, and images become links to where the image is stored.

- ☐ Build functions that run your documents through both Docling and MarkItDown.
- ☐ Preserve reading order: every heading, paragraph, table, formula, key-value pair, and image should appear where it did in the source.
- ☐ Save extracted images separately and link them from the Markdown (these links will point to S3 in Task 5).
- ☐ Render the final Markdown and check it side by side against the original PDF.
- ☐ Compare the two tools: what each gets right, what it drops or scrambles, speed, and ease of integration.

Done when: every input has a final Markdown file from each tool that renders cleanly and matches the original's structure, plus a short pros-and-cons comparison.

Task 5: Organize outputs in S3

Design and implement a storage layout that makes files easy to find later.

- ☐ Use a bucket name that's globally unique and describes its purpose.
- ☐ Use prefixes to separate files by type (raw PDFs, images, Markdown) and source (scraped site, uploaded document). Example: `raw/pdf/<source>/<date>/<file>.pdf`, `processed/markdown/<tool>/<source>/<file>.md`, `assets/images/<source>/<doc-id>/<image>.png`.
- ☐ Make sure Markdown files link to their images in S3.
- ☐ Add object metadata or tags (source URL, tool used, processing date, document type) for search and access control.
- ☐ Apply sensible partitioning, access controls (no public buckets by default), and encryption.

Done when: you can explain your naming scheme in a few sentences and anyone could find a given document's Markdown and images from it.

Task 6: Build the API and web app

Backend (FastAPI):

- ☐ Endpoints to upload a PDF or submit a URL, triggering the matching extraction pipeline.
- ☐ Endpoints to run Docling or MarkItDown standardization.
- ☐ Store processed files and metadata in S3 and return links to the results.

Front end (Streamlit):

- ☐ Let users upload a PDF or enter a URL, choose a tool, and view or download the Markdown.
- ☐ Connect it to your FastAPI backend.
- ☐ Deploy it on Streamlit Community Cloud, and host the backend somewhere publicly reachable.

Done when: a stranger can open your app link, submit a document, and get Markdown back.

What's required for submission

- ☐ **Working app link:** your live Streamlit app, plus the URL of your deployed FastAPI backend.
- ☐ **GitHub repository:** public, containing all your code and your three-way comparison write-up from Task 3.
- ☐ **README.md** in the repo covering:
 - **Architecture:** a diagram and short explanation of how the pieces fit together (extraction, standardization, S3 storage, API, front end).
 - **Step-by-step setup:** prerequisites, cloning the repo, installing dependencies, setting environment variables and credentials, running it locally, and deploying it.
- ☐ **Demo video:** about 5 minutes showing the app working end to end, from upload to final Markdown.

Ground rules

- Only scrape pages you're permitted to, and don't publish copyrighted content from your test set.
- Never commit API keys or AWS credentials. Use environment variables or a secrets manager.
- You are responsible for any cloud costs you incur while doing this challenge. The organizer will not cover or reimburse them, so be careful with how you use AWS, Azure, or any other platform. Done sensibly, the whole project should cost less than \$5: use free tiers and free credits where possible, set a billing alert or budget before you start, and shut down or delete resources when you're done.

Resources

- [Docling on GitHub](#)
- [MarkItDown on GitHub](#)
- [Azure AI Document Intelligence docs](#)
- [Amazon S3 security best practices](#)
- [FastAPI documentation](#)
- [Streamlit documentation](#)